

*Aprendizaje Automático · MCDA 2026-2*

## Taller — Olist: entregas, demanda y clientes

*Clasificación desbalanceada · Series de tiempo · Aprendizaje no supervisado · SQL con DuckDB*

**Tema:** un marketplace real de comercio electrónico visto desde tres problemas de ML

**Modalidad:** grupal

**Peso:** 20 % de la nota del curso

**Fecha de entrega:** sábado 17 de octubre de 2026, 11:59 p. m., vía Interactiva

**Datos:** [Brazilian E-Commerce Public Dataset by Olist](#) (Kaggle), cargado en DuckDB con el [script](#) del curso

**Entregables:** informe en PDF + notebooks (.ipynb) + consultas SQL

### 1. Contexto del problema

Olist es un marketplace brasileño que conecta pequeños vendedores con los grandes canales de venta en línea. El conjunto de datos público contiene cerca de 100 000 órdenes reales (anonimizadas) entre 2016 y 2018, con información de productos, vendedores, clientes, pagos, envíos y reseñas.

Un mismo negocio genera preguntas de naturaleza muy distinta, y cada una exige una familia diferente de métodos. En este taller usted trabajará como el equipo de datos de Olist y abordará tres de ellas:

1. **Operaciones:** ¿qué órdenes van a llegar tarde? clasificación desbalanceada
2. **Planeación:** ¿cuántas órdenes recibiremos en las próximas semanas? series de tiempo
3. **Mercadeo:** ¿qué tipos de clientes tenemos? aprendizaje no supervisado

El taller es **modular**: un módulo común de datos y cuatro módulos de modelado (A–D) que comparten la misma base de datos.

## 2. Datos y entorno de trabajo

---

### 2.1 Regla del entorno: todo pasa por SQL

En una empresa los datos rara vez llegan como un CSV limpio: viven en una bodega de datos y se acceden con SQL. Para simular ese entorno, el taller usa **DuckDB**, una base de datos analítica embebida que se instala con `pip install duckdb` y no requiere servidor.

**Obligatorio:** toda la lectura de datos debe hacerse con **consultas SQL** sobre el archivo `olist.duckdb`.

- No está permitido leer los CSV crudos (`pd.read_csv`, `read_csv` de DuckDB, Polars, etc.) en los notebooks.
- Los cruces entre tablas, los filtros, las agregaciones y las variables derivadas de agregados (conteos, promedios por vendedor, ventanas temporales) deben resolverse en SQL. Pandas se usa a partir del resultado de la consulta.
- Abra la conexión en modo de solo lectura (`read_only=True`): la base cruda no se modifica. Si necesita tablas intermedias, créelas en una base aparte o como vistas en memoria.
- Las consultas que dependan de una fecha de corte deben recibirla como parámetro (?) y no escribirla a mano.

## 2.2 Construcción de la base

El repositorio del curso incluye [homeworks/taller/build\\_olist\\_duckdb.py](#), que descarga el dataset desde Kaggle y crea `olist.duckdb` con una tabla por archivo. Las tablas se cargan **sin limpiar**: la validación es parte del trabajo.

### Construir la base

```
uv run python homeworks/taller/build_olist_duckdb.py --out data/olist.duckdb
```

### Plantilla sugerida de conexión

```
import duckdb

con = duckdb.connect("data/olist.duckdb", read_only=True)
print(con.sql("SHOW TABLES"))

# Parámetros con ? en lugar de f-strings
ordenes = con.execute("""
    SELECT o.order_id, o.order_purchase_timestamp, c.customer_state
    FROM orders AS o
    JOIN customers AS c USING (customer_id)
    WHERE o.order_purchase_timestamp < ?
""", ["2018-01-01"]).df()
```

## 2.3 Tablas disponibles

| Tabla                | Filas     | Contenido y llaves                                                                                                                                                    |
|----------------------|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| orders               | 99 441    | Una fila por orden: estado y cinco marcas de tiempo (compra, aprobación, entrega al transportador, entrega al cliente, fecha estimada). Llave <code>order_id</code> . |
| order_items          | 112 650   | Una fila por ítem: producto, vendedor, precio, flete y fecha límite de despacho.                                                                                      |
| order_payments       | 103 886   | Medio de pago, número de cuotas y valor. Una orden puede tener varios pagos.                                                                                          |
| order_reviews        | 99 224    | Calificación de 1 a 5, comentario y fechas de la reseña.                                                                                                              |
| customers            | 99 441    | Cliente por orden ( <code>customer_id</code> ), identificador único de persona ( <code>customer_unique_id</code> ), ciudad, estado y prefijo postal.                  |
| sellers              | 3 095     | Vendedor, ciudad, estado y prefijo postal.                                                                                                                            |
| products             | 32 951    | Categoría (en portugués), dimensiones, peso y número de fotos.                                                                                                        |
| geolocation          | 1 000 163 | Latitud y longitud por prefijo postal (con múltiples filas por prefijo).                                                                                              |
| category_translation | 71        | Traducción de las categorías al inglés.                                                                                                                               |

**Esquema y diccionario:** la [página del dataset en Kaggle](#) documenta cada columna e incluye el diagrama relacional. Revíselo antes de escribir la primera consulta.

## 3. Módulo 0 — Datos, calidad y EDA (común)

Este módulo alimenta a los demás. Todo debe resolverse con SQL y documentarse en el informe.

- **Granularidad y llaves.** Verifique la unicidad de cada llave y la cardinalidad de cada relación (1:1, 1:N). Identifique qué tablas multiplican filas al unirse y cómo evitarlo.
- **Calidad.** Valores faltantes, fechas inconsistentes (p. ej., entrega antes de la compra), órdenes sin ítems, pagos que no cuadran con precio + flete, prefijos postales sin geolocalización, categorías sin traducción.
- **Identidad del cliente.** Explique la diferencia entre `customer_id` y `customer_unique_id` y sus consecuencias para los módulos A y C.
- **Cobertura temporal.** Determine el período útil del dataset: hay meses al inicio y al final con muy pocas órdenes. Justifique qué rango usará.
- **EDA orientado a los módulos:** distribución de tiempos de entrega y del retraso, órdenes por día y por semana, distribución geográfica de clientes y vendedores, comportamiento de compra por cliente.

## 4. Módulo A — Clasificación: entregas tardías

*Al momento de aprobar una orden, ¿cuál es la probabilidad de que llegue después de la fecha estimada de entrega?*

**Población y variable objetivo.** Órdenes con `order_status = 'delivered'` y fecha de entrega al cliente no nula. El target vale 1 si `order_delivered_customer_date > order_estimated_delivery_date` (compare por día calendario) y 0 en otro caso. Reporte la proporción de la clase positiva.

**Fuga de información:** el modelo se usa en el **momento de la aprobación**. Ninguna variable puede depender de algo posterior: fecha de entrega al transportador, fecha real de entrega, reseñas, ni estadísticas históricas calculadas con órdenes que todavía no se habían entregado en esa fecha. Documente, para cada variable, por qué está disponible en ese momento.

**Partición temporal fija** (por `order_purchase_timestamp`), común a todos los grupos:

| Conjunto      | Compras entre           | Uso                                          |
|---------------|-------------------------|----------------------------------------------|
| Entrenamiento | 2017-01-01 y 2018-03-31 | Ajuste y validación cruzada temporal         |
| Validación    | 2018-04-01 y 2018-05-31 | Selección de modelo, balanceo y umbral       |
| Prueba        | 2018-06-01 y 2018-08-15 | Evaluación final, se usa <b>una sola vez</b> |

La tasa de retraso no es estable en el tiempo: cambia de forma notable entre entrenamiento, validación y prueba. Cuantifique ese cambio, discuta sus posibles causas y su efecto sobre las métricas y el umbral. La prueba termina el 15 de agosto porque las órdenes posteriores aún no se habían entregado cuando se tomó el dataset (censura).

### Tareas

- **Ingeniería de características (en SQL).** Ideas: distancia cliente–vendedor (a partir de geolocation), plazo prometido (estimada – compra), peso y volumen del paquete, flete relativo al precio, número de ítems y de vendedores, estado de origen y destino, categoría, medio de pago y cuotas, variables de calendario (día de la semana, festivos brasileños, cercanía a Black Friday), e históricos del vendedor y de la ruta (tasa de retraso previa) calculados *solo* con información anterior.
- **Baseline.** Un modelo ingenuo y una regla de negocio simple. Muestre por qué la *accuracy* no sirve aquí.
- **Desbalance.** Compare al menos dos estrategias (p. ej., `class_weight` o `scale_pos_weight`, sub-/sobre-muestreo dentro del pipeline, ajuste de umbral). El remuestreo nunca se aplica a validación ni a prueba.
- **Modelos.** Al menos tres familias (p. ej., regresión logística, Random Forest y un boosting moderno: XGBoost, LightGBM o CatBoost). Optimice hiperparámetros con `TimeSeriesSplit` u Optuna sobre el conjunto de entrenamiento y reporte media y desviación entre folds.
- **Métricas y umbral.** PR-AUC y ROC-AUC, curva precisión–recall, matriz de confusión en el umbral final. Proponga una matriz de costos (p. ej., costo de no avisar a un cliente que su orden llegará tarde

frente al costo de una intervención innecesaria) y elija el umbral que minimice el costo esperado. Evalúe la calibración de las probabilidades.

## 5. Módulo B — Series de tiempo: demanda diaria

---

*¿Cuántas órdenes recibirá Olist cada día durante las próximas 4 semanas?*

**Serie.** Número de órdenes por día según `order_purchase_timestamp`, construida con SQL (incluya los días sin órdenes). Como extensión opcional, modele también las 5 categorías con más ventas o la serie del estado de São Paulo.

**Partición.** Use como prueba las **últimas 8 semanas** del período útil que haya definido en el Módulo 0 y valide con *walk-forward* (origen móvil, horizonte de 28 días) sobre los datos anteriores.

### Tareas

- **Análisis de la serie.** Tendencia, estacionalidad semanal, ACF/PACF, estacionariedad (ADF/KPSS) y descomposición. Identifique y trate eventos atípicos (Black Friday 2017, festivos, paros) y justifique si los modela, los marca o los excluye.
- **Baselines.** Ingenio (último valor) e ingenio estacional (mismo día de la semana anterior).
- **Modelos.** Al menos un modelo estadístico (ETS, SARIMA/SARIMAX) y un modelo de ML con variables de rezago, medias móviles y calendario (p. ej., LightGBM). Construya los rezagos sin mirar al futuro y explique su estrategia multi-paso (recursiva o directa).
- **Métricas.** MAE, RMSE y MAPE o sMAPE por horizonte. Muestre cómo crece el error con el horizonte y compare contra los baselines.
- **Incertidumbre.** Reporte intervalos de predicción (del modelo estadístico, cuantílicos o conformales) y su cobertura empírica.

## 6. Módulo C — No supervisado: segmentación de clientes

---

*¿Qué grupos de clientes existen y qué acción comercial tiene sentido para cada uno?*

**Unidad de análisis.** El cliente real (`customer_unique_id`), con la información disponible hasta la fecha de corte **2018-06-01**.

### Tareas

- **Variables por cliente (en SQL).** RFM (recencia, frecuencia, valor monetario) y variables de comportamiento: ticket promedio, cuotas, medio de pago preferido, diversidad de categorías, flete relativo, calificación promedio, experiencia de entrega (retrasos sufridos) y región. Discuta qué ocurre con la frecuencia cuando la gran mayoría de los clientes compra una sola vez y cómo lo maneja.
- **Preprocesamiento.** Transformaciones para variables sesgadas (log, rangos), escalamiento y codificación. Justifique cada decisión.
- **Algoritmos.** K-means y al menos otro método de una familia distinta (jerárquico, GMM, DBSCAN/HDBSCAN). Elija el número de grupos con codo, silueta y Davies–Bouldin, y con criterio de negocio.
- **Visualización.** Proyección en 2D con PCA y UMAP o t-SNE, coloreada por segmento.
- **Estabilidad.** Compare las particiones entre semillas o submuestras (*Adjusted Rand Index*).
- **Perfilamiento.** Describa y nombre cada segmento con estadísticas por grupo y proponga una acción comercial concreta para cada uno.

## 7. Módulo D — Interpretabilidad e integración

---

- **SHAP sobre el Módulo A.** Importancia global (*beeswarm*), dependencia de las 3 variables más relevantes y explicación local de una orden correctamente detectada y de un falso negativo. Contraste con la importancia por permutación.
- **Integración.** Muestre cómo se conectan los módulos con al menos uno de estos análisis: (i) ¿qué segmentos del Módulo C sufren más retrasos?, (ii) ¿cuántas alertas diarias de retraso generaría el modelo A con la demanda proyectada por el Módulo B?, (iii) ¿mejora el modelo A al agregar variables del cliente?
- **Caso de uso.** Proponga cómo se operaría la solución (p. ej., alerta diaria de órdenes en riesgo, plan de capacidad semanal, campañas por segmento) y discuta sus limitaciones: deriva temporal, órdenes no entregadas al cierre del dataset, sesgos de cobertura geográfica.

## 8. Entregables

### 8.1 Informe en PDF

Máximo 20 páginas sin contar anexos. Debe incluir:

1. Resumen ejecutivo (una página, con las respuestas a las tres preguntas de negocio).
2. Datos, modelo relacional y decisiones de calidad (Módulo 0).
3. Un capítulo por módulo: formulación, variables, modelos comparados, validación, métricas y resultados.
4. Interpretabilidad, integración y caso de uso.
5. Limitaciones, conclusiones y trabajo futuro.

### 8.2 Notebooks y consultas SQL

- Un notebook por módulo (00\_datos, A\_clasificacion, B\_series, C\_segmentacion, D\_integracion) o uno solo bien organizado.
- Las consultas SQL principales también en archivos .sql dentro de una carpeta sql/, y leídas desde los notebooks.
- No entregue el archivo .duckdb: el calificador lo reconstruye con el script del curso.
- Código modular y comentado en español. Funciones reutilizables.
- Reproducibilidad: fije random\_state, declare versiones (requirements.txt o pyproject.toml) y use rutas relativas.
- Las celdas deben ejecutarse en orden y sin errores.

## 9. Rúbrica de evaluación

| Criterio                                  | Peso | Qué se evalúa                                                                                                                |
|-------------------------------------------|------|------------------------------------------------------------------------------------------------------------------------------|
| Módulo 0: SQL, calidad y EDA              | 15 % | Uso correcto de DuckDB, cruces sin duplicar filas, problemas de calidad detectados y resueltos, EDA que justifica decisiones |
| Módulo A: clasificación desbalanceada     | 25 % | Variables sin fuga, estrategias de balanceo, modelos y validación temporal, métricas, umbral por costo y calibración         |
| Módulo B: series de tiempo                | 20 % | Análisis de la serie, baselines, walk-forward, comparación de modelos, intervalos de predicción                              |
| Módulo C: segmentación                    | 15 % | Variables por cliente, preprocesamiento, selección del número de grupos, estabilidad y perfilamiento accionable              |
| Módulo D: interpretabilidad e integración | 15 % | SHAP bien leído, conexión entre módulos, caso de uso realista y limitaciones                                                 |
| Calidad del informe y del código          | 10 % | Redacción, organización, reproducibilidad, código limpio y comentado                                                         |